

Memoria del Proyecto Pay2Peer

1. Portada

- **Título:** Pay2Peer — Plataforma de transferencias y monedero
- **Autor/es:** Alejo Viñeta
- **Fecha:** 14/12/25
- **Versión:** 0.1

2. Resumen ejecutivo

- Este proyecto es una aplicación web de cartera y transferencia de dinero estilo PayPal con la estructura lógica de despliegue necesaria.
- El objetivo es crear un entorno seguro en el que tener una cartera de dinero y poder transferirlo entre carteras de forma segura y crear una infraestructura lógica donde alojarla.
- El objetivo es conseguir una aplicación que cumpla con los estándares legales de la Unión Europea sobre seguridad y privacidad en una aplicación financiera con la infraestructura lógica necesaria.

3. Índice

- 1. Portada
- 2. Resumen ejecutivo
- 3. Índice
- 4. Introducción
- 5. Objetivos
- 6. Requisitos
 - 6.1 Requisitos funcionales
 - 6.2 Requisitos no funcionales
 - 6.3 Criterios de aceptación
 - 6.4 Contratos de API
- 7. Arquitectura del sistema
 - Visión general
 - Componentes principales
 - Flujo de pagos
 - Modelos clave
- 8. Diseño e implementación
 - Frontend
 - Backend
 - Normalización de importes
- 9. Seguridad y privacidad
- 10. Despliegue
- 11. Conclusiones
- 12. Trabajo futuro

4. Introducción

- Este proyecto se inició para el curso de ASIR con especialización en ciberseguridad como proyecto de final de ciclo. La idea del proyecto viene de un proyecto de aprendizaje de React realizado anteriormente, en el que solo se creó un MVP con la funcionalidad básica para una demo. Es un proyecto que hasta ahora no he podido desarrollar más y este ciclo me ha dado la oportunidad de hacerlo, complementando mis conocimientos de desarrollo web con conocimientos de sistemas, redes y ciberseguridad para poder llevar este proyecto a término.
- El proyecto se divide en dos fases. La primera se entrega el 18/12/25 y tiene como alcance un MVP de la aplicación web. La segunda fase finaliza a finales de abril de 2026. Su alcance incluye securizar la aplicación según estándares europeos, preparar la infraestructura lógica necesaria para desplegar este proyecto y dejarlo listo para producción.

5. Objetivos

- **Objetivo general:** construir una aplicación segura y conforme a los requisitos legales para gestionar saldos y transferencias.
- **Objetivos específicos:** autenticación, persistencia de sesión, integración con Stripe, trazabilidad de transacciones y pruebas E2E.

6. Requisitos

6.1 Requisitos funcionales (RF)

- RF01 — Registro de usuario: `POST /api/users` → 201 + `{ user }`.
- RF02 — Autenticación: `POST /api/auth/login` → 200 + `{ token }`.
- RF03 — Persistencia de sesión: token en `localStorage`; `AuthContext` expone `token`, `user`, `logout` y `refreshUserAndWallet`.
- RF04 — Ver saldo y transacciones: `GET /api/wallets/:userId`, `GET /api/transactions`.
- RF05 — Recarga (pagos): el frontend solicita `POST /api/payments/create-payment-intent { amount }` y confirma con Stripe usando `clientSecret`.
- RF06 — Confirmación y contabilización: `POST /api/payments/confirm-payment-intent { paymentIntentId }` actualiza `Wallet` y crea `Transaction`.

6.2 Requisitos no funcionales (RNF)

- RNF01 — Seguridad: delegar la tokenización de tarjetas a Stripe; TLS en producción; no almacenar datos de tarjetas.
- RNF02 — Rendimiento: TTFB < 300 ms en staging para dashboard; creación de `PaymentIntent` < 3 s en condiciones normales.
- RNF03 — Disponibilidad: uso de colas para la persistencia de peticiones.

6.3 Criterios de aceptación (QA)

- CA01 — Flujo de recarga completo con tarjeta de prueba en Stripe.
- CA02 — Mensajes de error apropiados y sin exposición de datos sensibles.
- CA03 — Remount/retry del `CardElement` en errores `Element destroyed`.

6.4 Contratos de API (ejemplos)

- `POST /api/payments/create-payment-intent`
 - Request: `{ "amount": number }` (euros)
 - Response: `{ "clientSecret": string, "id": string, "amount": number }`
- `POST /api/payments/confirm-payment-intent`
 - Request: `{ "paymentIntentId": string }`
 - Response: `{ "success": boolean, "walletId?": string, "amount?": number }`

- `GET /api/payments/ready`
 - Response: `{ "ready": boolean }`

6.5 Restricciones

- No almacenar datos de tarjetas; usar Stripe Elements.
 - Secretos en variables de entorno (`STRIPE_API_KEY` , `JWT_SECRET`).
-

7. Arquitectura del sistema

Visión general

Cliente: Next.js (App Router) — Backend: Node.js/Express — BD: MongoDB — Servicios: Stripe.

Componentes principales

- Frontend: `app/` con `AuthContext` , `CheckoutForm` , `StripeClient` .
- Backend: `Pay2Peer-Node` con rutas de pagos en `src/resources/stripe` .
- Persistencia: colecciones `users` , `wallets` , `transactions` .

Flujo de pagos (resumen)

1. El frontend realiza un POST a `/api/payments/create-payment-intent` con `amount` .
2. El backend crea el `PaymentIntent` y devuelve `clientSecret` .
3. El frontend llama a `stripe.confirmCardPayment(clientSecret, { payment_method: { card } })` .
4. El backend verifica el resultado y actualiza `Wallet` y `Transaction` .

Modelos clave

- `User` : `_id` , `email` , `name` , `passwordHash` , `createdAt` .
 - `Wallet` : `_id` , `userId` , `funds` (string decimal), `currency` , `updatedAt` .
 - `Transaction` : `_id` , `walletId` , `type` , `amount` , `currency` , `stripePaymentIntentId` , `status` , `createdAt` .
-

8. Diseño e implementación

Frontend

- `app/context/AuthContext.tsx` : centraliza el token, el usuario y la wallet.
- `app/dashboard/fund/page.tsx` : `CheckoutForm` que consume `StripeClient` .
- `app/dashboard/fund/StripeClient.tsx` : monta Stripe Elements en cliente y maneja `remount/retry`.

Backend

- Endpoints principales: `create-payment-intent` , `confirm-payment-intent` , `ready` .
- `stripe.controller.js` : crea y verifica `PaymentIntents`; actualiza `Wallet` y `Transaction` .

Normalización de importes

- Usar céntimos (enteros) para llamadas a Stripe; almacenar en la BD como string/decimal con dos decimales.
-

9. Seguridad y privacidad

- Autenticación con JWT; Authorization: Bearer <token> en peticiones protegidas.
 - Validaciones y saneamiento de amount en frontend y backend.
 - No exponer STRIPE_API_KEY en el cliente.
-

10. Despliegue

Variables de entorno principales:

- Backend: STRIPE_API_KEY , JWT_SECRET , MONGO_URI .
- Frontend: NEXT_PUBLIC_API_ROOT , NEXT_PUBLIC_STRIPE_PK .

Build Frontend:

```
npm install
npm run build
npm run start
```

Build Backend:

```
npm install
npm run start
```

11. Conclusiones

- Se han alcanzado los objetivos de la primera fase, pero por restricciones de tiempo debidas a imprevistos no se ha podido implementar la seguridad ni la base de datos con una visión a futuro, por lo que será necesario refactorizar muchas partes en la segunda fase. Además, existe un bug errático en el formulario proporcionado por Stripe que no se ha conseguido depurar a tiempo para la primera fase.
- Ha habido una investigación sobre cómo securizar y desplegar correctamente el proyecto, por lo que se ha producido un aprendizaje en ese aspecto. Sin embargo, la primera fase del proyecto es meramente la infraestructura necesaria para poder empezar la parte más difícil del proyecto: la securización y la creación de la infraestructura de despliegue.

12. Trabajo futuro

- **Mejoras prioritarias:**
 - Testing.
 - Depuración del formulario de Stripe.
- **Nuevas funcionalidades planificadas:**
 - Implementación de blockchain en transacciones por seguridad.
 - Refactorización de la API para no exponer IDs de usuarios y carteras.
 - Uso de OAuth o SSO para el login con el fin de securizarlo.
 - Dockerización del proyecto.
 - Investigar e implementar nuevas medidas de seguridad acordes a los estándares de la Unión Europea.